

15-10

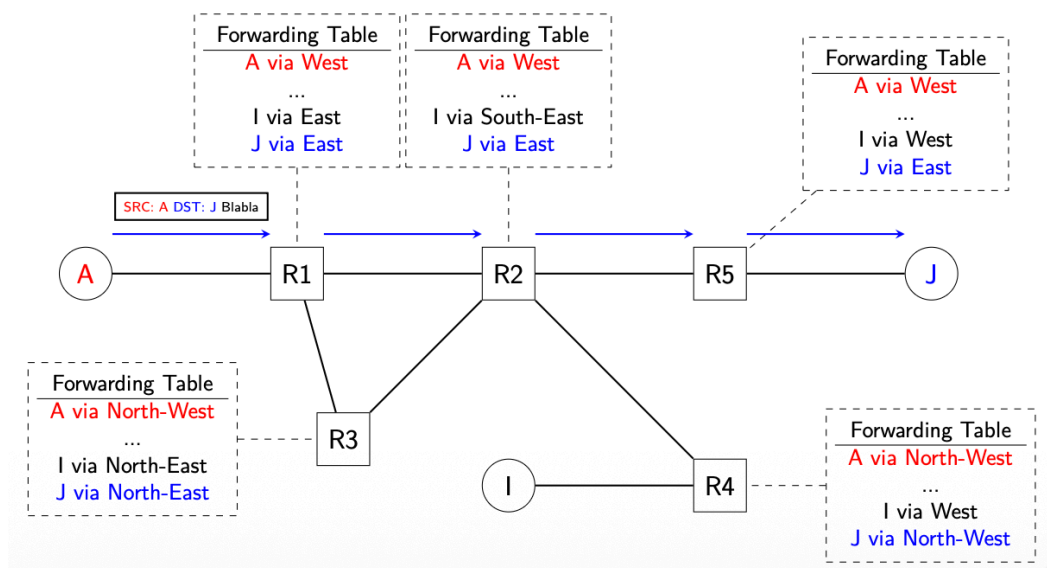
network layer

Il network layer assume un data link quasi affidabile: qualche perdita può avvenire, ma non frequentemente. Al network layer i frame diventano pacchetti. Un pacchetto contiene i dati + un network layer header, verrà incapsulato all'interno del payload del frame del datalink.

datagram-oriented network

- ogni host ha un indirizzo di rete
- per inviare informazioni a un host remoto si crea un pacchetto che contiene l'indirizzo di consegna e partenza e le informazioni
- i routers hanno hop by hop forwarding, se un router riceve un pacchetto che non doveva essere inviato a lui decide a chi mandare successivamente il pacchetto.

questa decisione è basata su una **FORWARDING TABLE**



deve essere consistente, ma possono accadere degli errori

black hole: un router dovrebbe mandare un pacchetto ma non ha un'entry nella sua forwarding table verso la destinazione.

routing loops: per arrivare a J R1 manda il pacchetto a R2 ma R2 lo rimanda a R1 che lo rimanderà a R2. I loop possono essere più lunghi di un hop.

il data plane: si occupa dei pacchetti in forwarding, a seconda della forwarding table.

il Control Plane: si occupa della creazione delle forwarding tables.

control plane

l'amministratore di rete scrive le di forwarding table a mano; questo funziona su reti piccole, nel caso di reti grandi i routers devono scambiarsi informazioni sulla loro visione locale della rete. può essere centralizzata attraverso un **controller** che monitora la rete fisica o distribuita

addressing flat

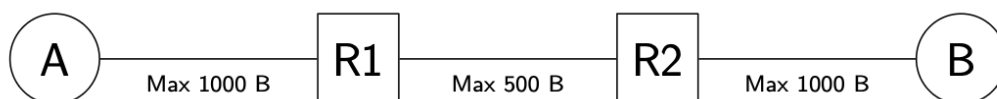
una tabella di forwarding deve contenere **tutte** le informazioni per raggiungere **tutte** le direzioni.

risulta più facile set upare la rete, ma non scala se una rete ha un milione di indirizzi, dovrà averne altrettanti anche la forwarding table, **performance degradation**.

indirizzo gerarchico

se l'host cambia rete cambia anche l'indirizzo, necessitiamo quindi di un modo per ricevere un indirizzo quando entriamo in una rete un modo per mantenere attiva una comunicazione cambiando l'indirizzo.

diversità tecnologica



R1 può fare 3 cose:

- rifiutare il pacchetto e inviarne uno di controllo indietro in cui dice che non può mandare i pacchetti superiori a 500B → a necessita di un buffer per i pacchetti inviati
- frammentare il pacchetto in due, poi mandati a R2 che riasssemblerà il pacchetto → forza i router a riassemble i pacchetti prima di inviarli, che introduce delay

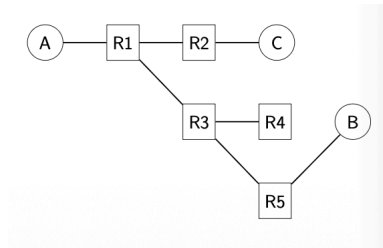


chi non può instradare frammenta, ma i frammenti vengono riassemblati a destinazione.

algoritmi di routing

hot potato (nominato così dal prof)

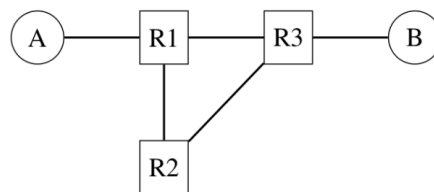
assumiamo che ognuno conosca solo se stesso e che A voglia mandare a B, A quindi manda a chiunque, R1 manda a tutti tranne ad A in perchè lo ha ricevuto da A: **network layer broadcast**



c quando ci riceverà il pacchetto, non avendo destinazioni a cui mandarlo lo scarta.

ogni invio del pacchetto comporta la modifica della forwarding table del ricevente del pacchetto con le informazioni del viaggio effettuato dal pacchetto stesso

si può fare solo con reti a ad albero in quanto in altri casi si può portare alla formazione di loops



quando R1 manda a R2 e a R3:

- R2 manda a R3 e R3 manda a B e a R2
- R2 manda a R1 anche dopo aver ricevuto da R3.



questo porta a un loop infinito e B continuerà a ricevere copie del pacchetto all'infinito

circuiti virtuali:

tabelle di routing

un routing a vettori di distanza usa un algoritmo di *Bellmann Ford distribuito*, un protocollo che lo implementa è il routing information protocol (RIP).

Lo scopo del control plane è costruire una routing table; ha 3 attributi:

- **R[d]. link** → link di forwarding per andare a d
- **R[d]. cost** → somma delle metriche della connessione che compone il percorso più breve per arrivare a d
- **R[d]. time** → tempo dell'ultimo controllo del pacchetto con informazioni di d

una routing table contiene più informazioni di una tabella di forwarding, e possono esserci più tabelle di routing per un lo stesso router. la fusione di tutte le routing table genera una forwarding table.

vettori di distanza

efficiente, facile da implementare, leggero ma lento a convergere.

assumiamo che ogni link abbia un costo: nel caso più semplice tutti i link hanno lo stesso costo unitario; assumiamo inoltre che ogni router contiene una routing table R con l'entry per se stesso , con distanza 0.

generazione DV

ogni tanto il router manda la propria routing table ai nodi vicini

```
1 # N : period of the generation
2 # R[] : the current routing table
3 Every N seconds:
4     v = Vector()
5     for d in R[]:
6         # add destination d to vector
7         v.add(Pair(d, R[d].cost))
8     for i in interfaces
9         # send vector v on this interface
10        send(v, i)
11
```

ricevendo un avviso:

```
1 # V : received Vector
2 # l : link over which vector is received
3 # R[] : the current routing table
4 def received(V, l):
5     # received vector from link l
6     for d in V[]:
7         if not (d in R[]):
8             # new route
9             R[d].link = l
10            R[d].cost = V[d].cost + l.cost
11            R[d].time = now()
12        else:
13            # existing route, should I update ?
14            if ((V[d].cost + l.cost) < R[d].cost) or (R[d].link == l):
15                R[d].link = l
16                R[d].cost = V[d].cost + l.cost
17                R[d].time = now()
18
```



i pacchetti non sono mai forwarded, non accade mai che un router broadcasti un pacchetti che ha ricevuto da un'interfaccia ai vicini, ogni router riceve un DV aggiorna la sua routing table e quando il suo timer termina manda il suo DV ai vicini
a un certo punto le routing tables possono essere sub-ottimali, alla fine arriveranno a uno stato ottimale ma possono comprendere delle fasi intermedie.

